

20 Watts: An Honest Audit of Brain-Inspired LLM Efficiency

The 20 Watts series — overview paper

[Your Name], age 17 — June 2026 · github.com/svaka2000/20-watts

Abstract

The human brain performs general intelligence on roughly **20 watts**; a large language model needs a rack of accelerators. A popular framing blames *storage precision* — the brain stores fuzzy memories, so quantize the weights. That is true but incomplete. The brain's deeper efficiency strategies are about **doing less computation**: firing almost no neurons, predicting away the expected, and remembering only the gist. This series takes four such principles, maps each to a concrete inference-time lever, and measures it honestly on a single real model — **Qwen2.5-7B-Instruct (4-bit)** — on a **laptop** (Apple M4 Pro, MLX), with a bit-exact harness so that every intervention is verified not to silently change the model. We report wins, non-wins, and a synthesis. The point is not a single hero number; it is an honest map of which brain tricks actually transfer.

The framework

Brain principle	Episode	Mechanism	Lever	Result on Qwen2.5-7B
Sparse firing (<1% active; Lennie 2003)	1	top-k MLP neuron skipping	computation	skip 60% of neurons → <1% ppl; ~52% compute @ <1% quality
Predictive coding (spend on surprise; Rao–Ballard 1999)	2	adaptive depth / early exit	depth	n nuanced: only ~1 layer freely droppable; per-token headroom needs a calibrated exit head
Foveated/gist memory	3	KV sinks + sliding window	memory	constant memory; +34% ppl at 74% memory saved; attention-sink: –sink → +542%
Low-resolution storage	(the "other guy")	4-bit quantization	storage	4× smaller weights (orthogonal, already applied)

Episode 1 — Sparse Firing (the big win)

On every token a dense transformer fires 100% of its MLP neurons. We find ~89% of the activation magnitude lives in the top half of neurons, and that **skipping 60% of MLP neurons per token changes held-out perplexity by < 1%** (at 50%, perplexity *improves*). Since the MLP is **87% of the per-token compute**, that is a **~52% inference-compute reduction at <1% quality cost**, verified bit-exact, and it holds on a downstream task (ARC-style MCQ accuracy is flat under sparsity). A small trained predictor recovers most of the activation *mass* of the active set, showing the saving is realizable in

the spirit of Deja Vu — strongly in early/late layers, with middle layers needing a stronger predictor. **This is the robust, free lever.**

Episode 2 — Predictive Coding (the honest non-win)

If easy tokens didn't need full depth, we could exit early. We measured this two ways. **Static:** dropping whole layers shows this model is *not* very depth-redundant — only about one layer is freely removable and the final layers are critical (a useful corrective to "just delete layers" claims). **Per-token:** a raw logit lens says predictions stabilize late, but that is a known lens artifact; a **tuned lens** (per-layer calibrated probes) gives the true adaptive-depth headroom (*see results/tuned_lens_results.json*). The honest takeaway: depth is the lever that does **not** hand you a free lunch on this model — and saying so is the difference between science and a hype reel.

Episode 3 — Foveated Memory (a discovery)

Keeping only a few initial "sink" tokens plus a recent window makes memory and attention cost **constant** regardless of context length. The quality cost is real and bounded (keep 26% of the cache → +34% perplexity). The striking result is the **attention sink**: removing the first few tokens explodes perplexity by +542%, and a *single* sink token recovers almost all of it — a vivid, reproduced example of a mechanism a model secretly relies on, caught with a faithfulness-checked harness. And at a fixed budget, keeping high-attention *heavy hitters* (H2O) beats pure recency (+69% vs +79% perplexity at 92% eviction) — verified with a bit-exact manual-attention implementation.

Synthesis — what stacks

Because the four levers act on different resources (storage, computation, depth, memory) they compose. The robust, quality-preserving stack is **4-bit + sparse firing: 4× smaller weights and ~35% less compute at +0.3% perplexity**, measured on one model. Pushing depth and memory harder buys more (−37% compute, −60% KV memory) at a real quality cost (+6% to +68%), to be spent only when the budget demands it.

Why this is credible (and why the reference video isn't)

- **Bit-exact harness.** Every intervention is checked against the unmodified model ($\max |\Delta| = 0$) before any claim is made.
- **Paired numbers.** Every compute/memory figure is reported with its quality cost — never a lone "10×."
- **Honest non-results.** Episode 2 reports a non-win. Real audits have them.
- **Reproducible on a laptop.** One command per experiment; open code and data.

Methods

All experiments use `mlx-community/Qwen2.5-7B-Instruct-4bit` via MLX on an Apple M4 Pro. Perplexity is on held-out WikiText-2 / original prose; the downstream task is ARC-Easy. Predictors and the tuned lens are trained in MLX. See each episode's paper and `src/` for exact protocols.

Selected references

Lennie 2003; Attwell & Laughlin 2001; Li et al. 2023 (Lazy Neuron, ICLR); Liu et al. 2023 (Deja Vu, ICML); Mirzadeh et al. 2024 (ReLU Strikes Back, ICLR); Rao & Ballard 1999; Schuster et al. 2022 (CALM, NeurIPS); Belrose et al. 2023 (Tuned Lens); Xiao et al. 2023 (StreamingLLM); Zhang et al. 2023 (H2O, NeurIPS).